

如何从零开始将神经网络移植到FPGA(ZYNQ7020)加速

于 2021-05-30 13:59:30 发布



FPGA proj 专栏收录该内容

1 篇文章

订阅专栏

如何从零开始将神经网络移植到FPGA(ZYNQ7020)加速推理

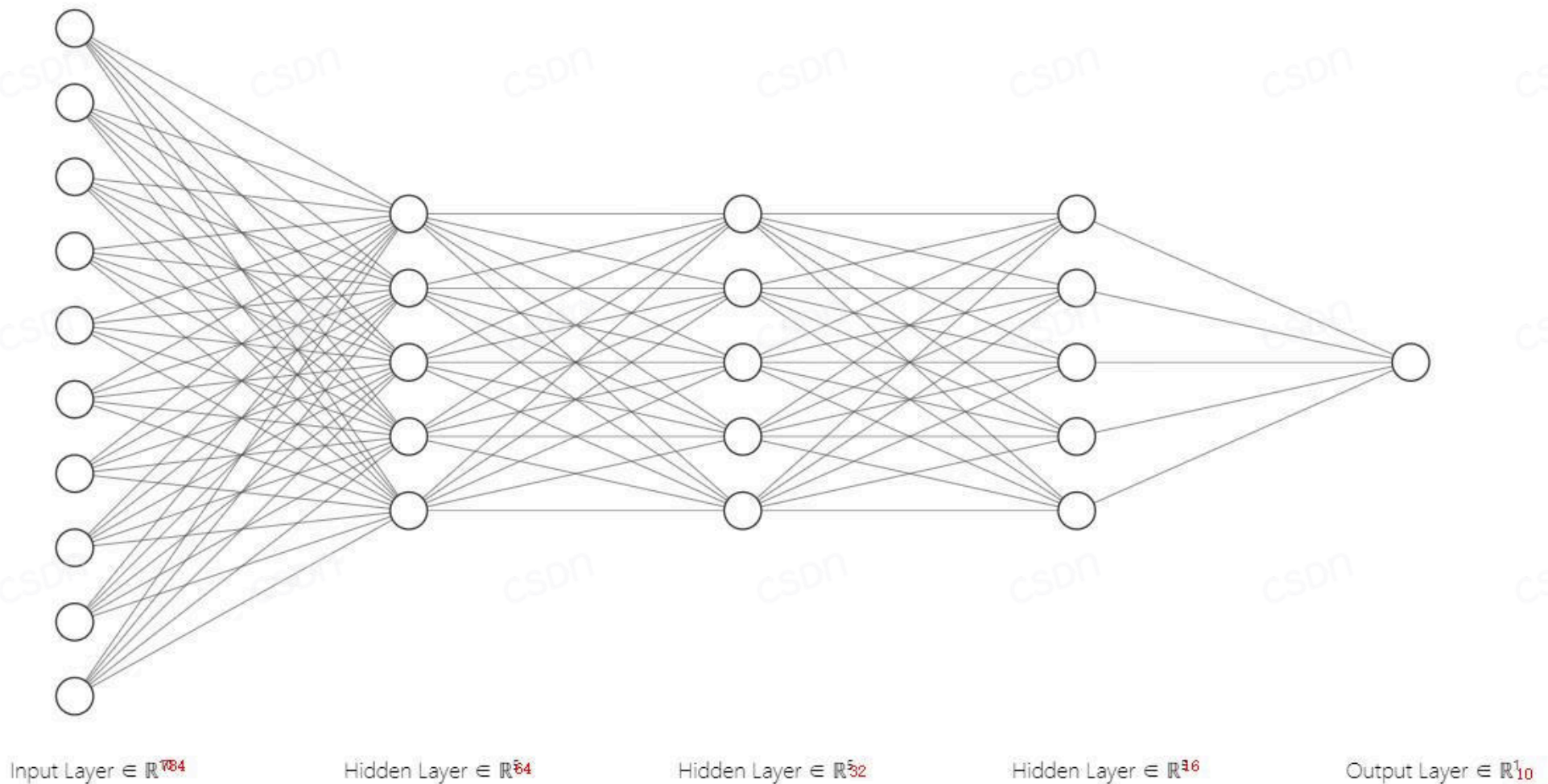
前言

本片文章用于对零基础的小白使用，仅供参考，大神绕道。AI 一直都是做算法的热点，作为多少研究生都想蹭一蹭热度，本文就神经网络的移植到FPGA做一个简单的教程。使用FPGA做前向的推理而不是训练，训练还是在PC机上完成的，通过训练得到的权重文件然后再去移植到FPGA。

1.神经网络

深度神经网络应该是最简单的一种网络结构了，相比CNN没有卷积相关的参数，仅仅只有几层网络结构，深度神经网络就不多说了，DNN，之所以不多说的原因就是这个东西应该很容易理解，毕竟不是很难的算法，现在前沿的算法估计都没人去研究DNN相关的了。不懂的可以上网去搜索相关的原理包括、前向传播、反向传播、梯度下降等等。既然要移植神经网络，总要有一个神经网络了吧。既然是入门那就用入门的鼻祖—手写字了，首先需要弄一个手写字的神经网络。

内容来源: csdn.net
作者昵称: Jarvis码员
原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>
作者主页: <https://blog.csdn.net/u012116328>



很简单的三层

1.1 如何规划神经网络的结构

- 1.神经网络的结构可以自己规划，没必要和本文的一模一样，也可以使用主流的框架去做，比如`tensorflow`等，本文使用三层隐藏层网络，输入层为784，隐藏层1为64，2为32，3为16。三层网络可做到识别率95%以上。
- 2.其实网络刚开始用的隐藏层为256、128、32，这样的识别精度更高，能达到98%，但是后面去移植到FPGA中发现权重参数太多了，FPGA的Bram资源远远不够用，

所以对网络的结构进行删减。把网络结构调整的简单了一下，再FPGA中能够实现了，精度也不至于削减的这么多，足够了。

3.激活函数用的sigmoid函数，这个也不用多说，了解过的应该都知道是臭了大街的了。一个神经网络从输入输出都要自己设定好。本文的输入2828，输出101.对应每一层的权重数量为[64][784]、[64][64]、[10][32]、偏置项[64]、[32]、[10]。

4.保存到数据类型采用的是float类型数据，每个权重按照4字节存储。

1.2mnist数据集

这部分数据集没什么好说的，使用官网的文件解析出来把数据读取，然后使用数据集去训练。只不过注意的是，在使用过程中，将数据集中的参数提取了出来保存为.dat格式的文件，便于使用FPGA预测单张图片。另外就是每张图片的参数原始参数是0-255，最后都归一化到了0-1之间。

1.3 有关权重文件的保存

本文中的权重文件分开保存，对每一层的weight、bias等文件分别保存成独立的文件，weightx.dat与biasx.dat,之所以这样保存的原因是使用HLS Q语言开发时，可以使用#include参数读取权重文件等。当然，对在PC上完成的训练与测试，输出保存的权重文件只有保存了1个文件。这个可以在源代码中查看，本文全部开源。相关代码可以从本文链接下载也以从github下载。

2.FPGA实现

FPGA实现是一个重要的地方，本出不主要是去探究实现神经网络的各种方法。个人觉得，FPGA去实现神经网络最关键的地方就是设定好一种架构，包括数据的传输、临时存储、结果读取。比如，我PS做控制，调取PL资源做网络推理，那么PS与PL之间的交互九世纪最重要的，可以通过AXI协议、可以通过BRAM等等，这样就要考虑板子的资源够不够用，一个好的架构不同的方式必然会带来不一样的效果。本文中实现使用HLS完成的加速IP核设计，实现ARM+HLS调用的硬件结构。

2.1HLS加速IP核设计

HLS直白了就是直接使用高级语言描述硬件电路，不得不说这种方式开发真的是省时间效率，但是综合出来的电路质量是差一些，后期优化一下可能会更好，一下代码没有经过优化，使用流水线优化下FOR循环效果会更好，废话不多说直接上代码：

```
1  /**
2   * Use the saved CNet model to predict over the MNIST dataset. compile by hls
3   **/
4  #include <math.h>
5  #include "hls_mnist.h"
6  /**
7   * Sigmoid
8   *
9   * @param double *: Sum of weights * inputs + Bias
10  * @param int: Size
11  */
```

内容来源：csdn.net

作者昵称：Jarvis码员

原文链接：<https://blog.csdn.net/u012116328/article/details/117246023>

作者主页：<https://blog.csdn.net/u012116328>

```

12 */
13 void Sigmoid( float *a,int size)
14 {
15     for(int i = 0; i < size; i++)
16         a[i] = 1/(1 + expf(-a[i]));
17 }
18
19 void mnist_nn_predict( float input[784],float output[10])
20 {
21     #pragma HLS INTERFACE s_axilite port=return bundle=CRTL_BUS
22     #pragma HLS INTERFACE bram port=input
23     #pragma HLS INTERFACE bram port=output
24     int max_idx = 0;
25     // initialize the weight matrix
26     const float weight1[64][784] = {
27         #include "weight1.dat"
28     };
29     const float weight2[32][64] = {
30         #include "weight2.dat"
31     };
32     const float weight3[10][32] = {
33         #include "weight3.dat"
34     };
35     // initialize the bias matrix
36     const float bias1[64] = {
37         #include "bias1.dat"
38     };
39     const float bias2[32] = {
40         #include "bias2.dat"
41     };
42     const float bias3[10] = {
43         #include "bias3.dat"
44     };
45     // initialize the confusion matrix
46     int k ,j,i;
47     float output1[64];
48     float output2[32];
49     float output3[10];
50     float z1 = 0.0;

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

51 float z2 = 0.0;
52 float z3 = 0.0;
53 // pass through every layer in the net
54 // pass through every neuron in the net
55 //the first layer
56 for( k = 0; k < 64; k++)
57 {
58     z1 = 0.0;
59     // compute z for neuron
60     for( j = 0; j < 784; j++)
61         z1 += weight1[k][j] * input[j];
62     z1 += bias1[k];
63     output1[k] = z1;
64 }
65 Sigmoid(output1,64);
66 //the second layer
67 for( k = 0; k < 32; k++)
68 {
69     // compute z for neuron
70     z2 = 0.0;
71     for( j = 0; j < 64; j++)
72         z2 += weight2[k][j] * output1[j];
73     z2 += bias2[k];
74     output2[k] = z2;
75 }
76 Sigmoid(output2,32);
77 //the third layer
78 for( k = 0; k < 10; k++)
79 {
80     // compute z for neuron
81     z3 = 0.0;
82     for( j = 0; j < 32; j++)
83         z3 += weight3[k][j] * output2[j];
84     z3 += bias3[k];
85     output3[k] = z3;
86 }
87 Sigmoid(output3,10);
88 for( k = 0; k < 10; k++)
89     output[k] = output3[k];
90

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

90     for( i = 0; i < 10; i++) {
91         if(output[i] > output[max_idx]) {
92             max_idx = i;
93         }
94     }
95 }

```

一键获取完整项目代码 `javascript`

1. 简简单单的三层网络，最后输出结果为10个分类结果0-9，当然稍加修改可以直接获取单个识别的结果。
2. 开头的前三行是把HLS入参综合成BRAM接口，之所以设计成BRAM的接口是因为调用IP核时，可以直接访问BRAM的地址获取数据，当然也可以综合成AXI_slave类型的接口，具体大家可以去试一下，至于返回值综合成了AXI的接口。
3. 对于for循环部分大家可以使用 directive 相关参数优化加入流水线去改变延时，源码中的没有经过优化，延时非常大，加入了流水线优化是非常明显的，这个可以自行去试一下。
4. #include用法可以直接将文件的参数加载到数组中，这个对于HLS编译来说是可以执行的，这种用法大家可以百度下GCC-E相关 的解释，预处理文件。

3.ARM调用IP核

3.1 硬件设计

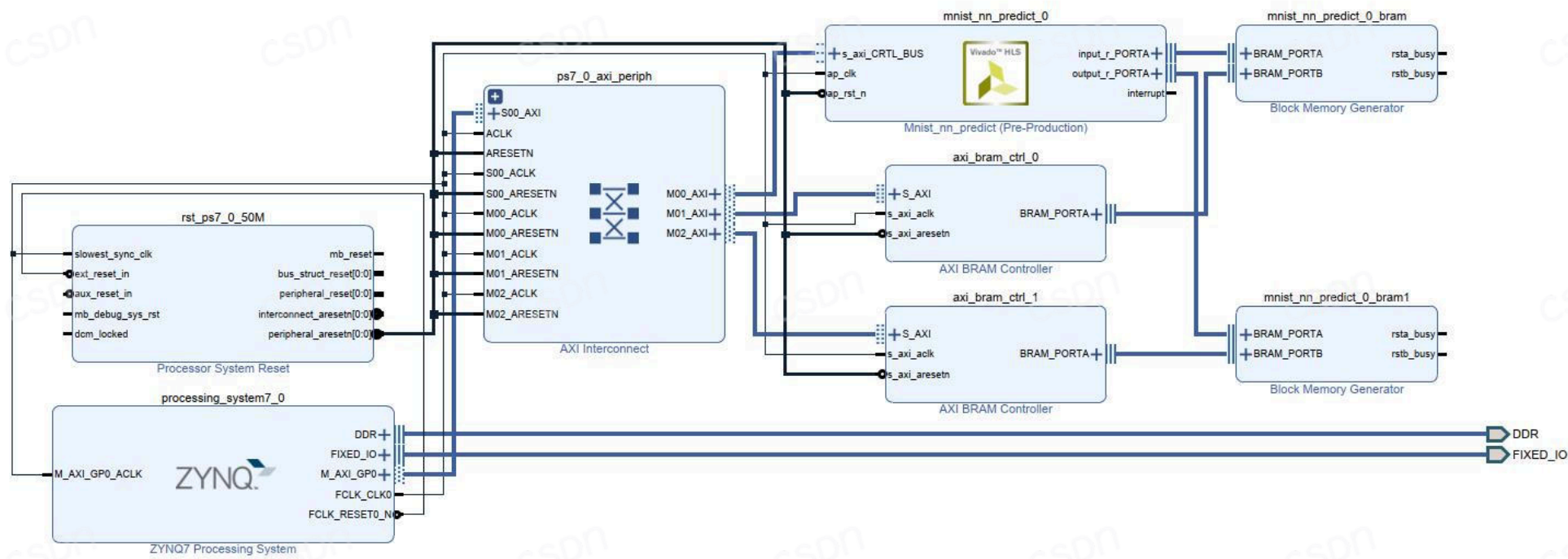
如图

内容来源: [csdn.net](https://blog.csdn.net/)

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>



<https://blog.csdn.net/u012116328>

Diagram x Address Editor x

</

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

1.首先添加ps处理器硬件，使能串口（用于调试输出）、SD卡引脚（用于从读取手写字数据文件）、axi GP master 等

2.添加HLS的手写字识别 IP核

3.添加BRAM1，双端口，添加BRAM控制器1，两个端口分别连接HLS的IP的INPUT端接口、和BRAM控制器1，BRAM控制器1链接PS处理器，该处Bram1用于存放输入数据，该处的数据是PS端从SD卡读取手写字数据文件后写入，然后HLS IP核从该处读取手写字数据识别计算。。根据实际存放的数据可以计算出该BRAM1的大小，根据计算出来的大小再分配BRAM的。地址深度等。此处的地址深度大于等于手写字图片的数据量。

4添加BRAM2，双端口，添加BRAM控制器2，一个端口链接HLS IP核的输出端OUTPUT，用于存放IP核识别结果。共计10个数；一个端口通过BRAM控制器2连接到PS端，用于PS端读取输出结果串口打印输出。以上便完成了整体的硬件设计。

3.1SDK裸机调用IP核

设计完硬件电路，便开始使用SDK写应用函数。对于PL资源来说，算是PS的一个外设，再使用的时候也是，把PL当成PS的其中一个外设，通过寄存器地址读取就可以了。

由于需要使用SD卡存取手写字，所以需要使使用fatfs文件系统，在创建SDK应用的时候，需要将文件系统添加到BSP中。

```
1 //*****Copyright (c)*****//
2
3 // Created date:      2019/10/8 17:25:36
4 // Version:           V1.0
5 // Descriptions:      The original version
6 //
7 //-----
8 //*****//
9 #include "xparameters.h"
10 #include "xil_printf.h"
11 #include "ff.h"
12 #include "xdevcfg.h"
13 #include <stdio.h>
14 #include <stdlib.h>
15 #include "platform.h"
16 #include "xil_printf.h"
17 #include "xil_io.h"
18 #include "xmnist_nn_predict.h"
19 #define FILE_NAME "1.dat" // 定义文件名
20 const char src_str[30] = "www.openedv.com"; // 定义文本内容
21 static FATFS fatfs; // 文件系统
22
23 u32 float_to_u32(float value)
24
```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>


```

25 {
26     u32 result;
27     union float_byte
28     {
29         float v;
30         u8 byte[4];
31     }data;
32     data.v =value ;
33     result = (data.byte[3]<<24)+(data.byte[2]<<16)+(data.byte[1]<<8)+(data.byte[0]<<0);
34     return result;
35 }
36 float u32_to_float(u32 value)
37 {
38     return *((float*)&value);
39 }
40 // 初始化文件系统
41 int platform_init_fs()
42 {
43     FRESULT status;
44     TCHAR *Path = "0:/";
45     BYTE work[FF_MAX_SS];
46     // 注册一个工作区( 挂载分区文件系统)
47     // 在使用任何其它文件函数之前, 必须使用f_mount函数为每个使用卷注册一个工作区
48     status = f_mount(&fatfs, Path, 0); // 挂载SD卡
49     if (status != FR_OK) {
50         xil_printf("Volume is not FAT formated; formating FAT\r\n");
51     }
52     return 0;
53 }
54 // 挂载SD(TF) 卡
55 int sd_mount()
56 {
57     FRESULT status;
58     // 初始化文件系统 (挂载SD卡, 如果挂载不成功, 则格式化SD卡)
59     status = platform_init_fs();
60     if(status){
61         xil_printf("ERROR: f_mount returned %d!\n",status);
62         return XST_FAILURE;
63     }

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

64     return XST_SUCCESS;
65 }
66 //SD卡写数据
67 int sd_write_data(char *file_name,u32 src_addr,u32 byte_len)
68 {
69     FIL fil;          //文件对象
70     UINT bw;          //f_write函数返回已写入的字节数
71
72     //打开一个文件,如果不存在,则创建一个文件
73     f_open(&fil,file_name,FA_CREATE_ALWAYS | FA_WRITE);
74     //移动打开的文件对象的文件读/写指针    0:指向文件开头
75     f_lseek(&fil, 0);
76     //向文件中写入数据
77     f_write(&fil,(void*) src_addr,byte_len,&bw);
78     //关闭文件
79     f_close(&fil);
80     return 0;
81 }
82 //SD卡读数据
83 int sd_read_data(char *file_name,u32 src_addr,u32 byte_len)
84 {
85     FIL fil;          //文件对象
86     UINT br;          //f_read函数返回已读出的字节数
87
88     //打开一个只读的文件
89     f_open(&fil,file_name,FA_READ);
90     //移动打开的文件对象的文件读/写指针    0:指向文件开头
91     f_lseek(&fil,0);
92     //从SD卡中读出数据
93     f_read(&fil,(void*)src_addr,byte_len,&br);
94     //关闭文件
95     f_close(&fil);
96     return br;
97 }
98 //main函数
99 int main()
100 {
101     FIL fil;          //文件对象
102     int flielen,br;
103

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

103 char dest_str[5000] = "";
104 char dest_str0[5000] = "";
105 char dest_str1[784][10] = {{0}};
106 float f_buff; //teapota float
107 char buff[10] = {0};
108 float data[784] = {0};
109 char *p;
110 u32 result, revs;;
111 int status, len, i, k;
112 int n = 0;
113 // init_platform();
114 Xil_DCacheDisable();
115 //scleanup_platform();
116 printf("sd_mount start\n");
117 // 挂载SD卡
118 status = sd_mount();
119
120
121 if(status != XST_SUCCESS)
122 {
123     printf("sd_mount failed\n");
124     return 0;
125 }
126 //get the file from sd
127 f_open(&fil, FILE_NAME, FA_READ);
128 // 移动打开的文件对象的文件读/写指针 0: 指向文件开头
129 f_lseek(&fil, 0);
130 // read file
131 f_read(&fil, (void*)dest_str, 5000, &br);
132 //to devide str to several numeral str
133 p = strtok(dest_str, ",\n\r");
134 memcpy(dest_str1[0], p, strlen(p));
135 i = 1;
136 // 继续获取其他的子字符串
137 while( p != NULL )
138 {
139     p = strtok(NULL, ",\n\r");
140     memcpy(dest_str1[i++], p, strlen(p));
141 }
142 //tran str to float

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

143     for(i=0;i<784;i++)
144     {
145         data[i] = atof(dest_str1[i]);
146
147         //printf("dest_str1[i]:%f\n",data[i]);
148     }
149     printf("data show:\n");
150     for( k=0;k<28;k++)
151     {
152         for( i=0;i<28;i++)
153         {
154             printf("%.0f",data[n++]);
155         }
156         printf("\n");
157     }
158     for(i=0;i<784;i++)
159     {
160         result = float_to_u32(data[i]);
161         Xil_Out32(XPAR_AXI_BRAM_CTRL_0_S_AXI_BASEADDR+i*4,result);
162     }
163     //Init the pl_neron net
164     XMnist_nn_predict HlsXMem_test;
165     XMnist_nn_predict_Config *ExamplePtr;
166     printf("Look Up the device configuration.\n");
167     ExamplePtr = XMnist_nn_predict_LookupConfig(XPAR_MNIST_NN_PREDICT_0_DEVICE_ID);
168     if (!ExamplePtr)
169     {
170         printf("ERROR: Lookup of accelerator configuration failed.\n\r");
171         return XST_FAILURE;
172     }
173     printf("Initialize the Device\n");
174     status = XMnist_nn_predict_CfgInitialize(&HlsXMem_test, ExamplePtr);
175     if (status != XST_SUCCESS)
176     {
177         printf("ERROR: Could not initialize accelerator.\n\r");
178         return(-1);
179     }
180     XMnist_nn_predict_Start(&HlsXMem_test);
181     while (XMnist_nn_predict_IsDone(&HlsXMem_test) == 0);
182

```

内容来源: csdn.net

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>

```

182     for(i=0;i<10;i++)
183     {
184         revs=Xil_In32(XPAR_AXI_BRAM_CTRL_1_S_AXI_BASEADDR+4*i);
185         float recvf = u32_to_float(revs);
186         printf("recongize result:%f\n",recvf);
187     }
188     printf("all vision over\n");
189     cleanup_platform();
190     return 0;
}

```

一键获取完整项目代码

javascript

整个代码流程如上，需要

- 1.初始化平台参数，
- 2.初始化挂载SD卡，前提是SD卡已经格式化为fat32文件格式，如果 SD没有被格式，可以使用fatfs的f_mkfs函数格式化SD卡
- 3.初始化查找表，调用HLS驱动函数。HLS生成RTL IP核文件后会自动生成驱动函数，生成的驱动函数以及相关的调用方式都是固定的，基本上就是初始化相关结构体，然后调用函数，获取结果，相关的驱动函数可以去SDK头文件中查找。
- 4.调用驱动函数获取结果

注意点：xlinx驱动函数传递的数据一般都使用u32数据类型，也就是unsigned int ,在使用SDK相关参数传递的时候对于浮点型数据float要转成unsigned int 进行传输，具体原理大家可以看ieee 754标准。

结果图

内容来源：csdn.net

作者昵称：Jarvis码员

原文链接：<https://blog.csdn.net/u012116328/article/details/117246023>

作者主页：<https://blog.csdn.net/u012116328>

内容来源: [csdn.net](https://blog.csdn.net/u012116328)

作者昵称: Jarvis码员

原文链接: <https://blog.csdn.net/u012116328/article/details/117246023>

作者主页: <https://blog.csdn.net/u012116328>